

SDR & FPGA Engineering Transition

Skill Acquisition Program for Defense Autonomous Systems

This curriculum provides a structured, hands-on path to acquire the specific DSP and FPGA skills required for advanced embedded defense platforms. The program is divided into five sequential phases, moving from high-level software signal processing down to bare-metal logic gates, culminating in a production-ready hardware portfolio project.

Phase 1: SDR & DSP Software Fundamentals

Before designing digital hardware, it is critical to master signal processing in the software domain. This phase bridges mathematical theory with applied telecommunications, focusing on the ingestion, processing, and modulation of real-world I/Q data streams.

- **Objective:** Build software flowgraphs and scripts to capture, demodulate, and filter real-world RF signals.
- **Key Concepts:** Nyquist-Shannon sampling, FFTs, QPSK/QAM modulation, FIR/IIR filtering.
- **Hardware Needed:** An RTL-SDR dongle (approx. \$30) for data ingestion.

Concept	Mathematical Function	Applied SDR Use-Case
Digital Mixing	Multiplication by a complex sinusoid	Frequency shifting signals to baseband for analysis.
Decimation	Low-pass filter followed by downsampling	Reducing high sample rates to manageable computational loads.
FIR Filtering	Discrete convolution	Isolating specific transmission bands and removing adjacent noise.
FFT	Time domain to frequency domain conversion	Visualizing the RF spectrum and identifying active signal peaks.

PHASE 1 REQUIRED RESOURCES

PySDR (pysdr.org): A comprehensive, free online guide to SDR and DSP using Python. Complete the chapters up through Pulse Shaping and Synchronization.

GNU Radio (gnuradio.org): Download the framework and complete the official "Guided Tutorials" to understand block-based signal flows.

Phase 2: HDL & FPGA Basics

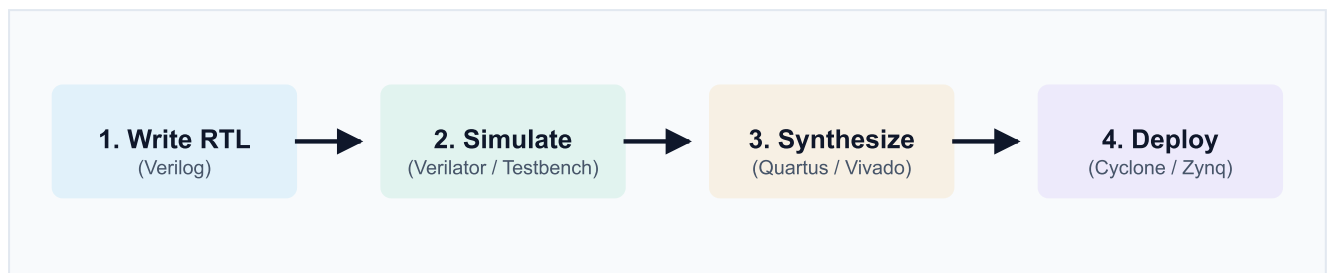
Field Programmable Gate Arrays (FPGAs) do not execute software; they physically wire logic gates. This phase requires a paradigm shift from sequential programming (C++/Python) to parallel hardware description using Verilog.

- **Objective:** Master clock domains, state machines, and writing testbenches for logic verification.
- **Key Concepts:** Combinational vs. sequential logic, Verilog syntax, simulation.
- **Hardware Needed:** Intel Cyclone IV or V development board (e.g., DE10-Nano).

PHASE 2 REQUIRED RESOURCES

HDLBits (hdlbits.01xz.net): An interactive, web-based Verilog practice platform. Complete up to the "Finite State Machines" section.

Verilator (verilator.org): The industry-standard open-source Verilog simulator. Learn to write testbenches in C++ to verify your Verilog code before deploying to hardware.



Phase 3: Hardware-Accelerated DSP

This phase merges the knowledge of Phase 1 and Phase 2. You will implement deterministic, real-time signal processing modules directly in Verilog hardware, replacing the slower Python/C++ implementations.

- **Objective:** Build functional DSP blocks (mixers, NCOs, FIR filters) in RTL.
- **Key Concepts:** Direct Digital Synthesis (DDS), pipelining, mathematical hardware instantiation, fixed-point arithmetic.
- **Resource:** The **ZipCPU Blog (zipcpu.com)** is an unparalleled resource for learning how to structure DSP algorithms specifically for FPGA logic, including detailed guides on building hardware FIR filters and PLLs.

Phase 4: SoC Architecture & System Integration

Modern defense systems require complex data routing between hardware logic and host operating systems. This phase focuses on Systems-on-Chip (SoC) where the FPGA fabric interfaces with embedded ARM processors over high-speed buses.

- **Objective:** Interface FPGA logic with CPU systems via streaming interfaces and Direct Memory Access (DMA).
- **Key Concepts:** AXI4-Stream protocols, Yocto/Buildroot for embedded Linux, memory mapping.
- **Hardware Needed:** SoC board (Analog Devices PlutoSDR or DE10-Nano).

PHASE 4 REQUIRED RESOURCES

RocketBoards (rocketboards.org): The definitive resource for integrating embedded Linux with Intel (Cyclone) SoCs.

Yocto Project Mega-Manual: Learn how to bake custom embedded Linux distributions tailored to your hardware architecture.

Phase 5: The Capstone Portfolio Project

To demonstrate readiness for a defense-grade engineering role, you must build a comprehensive, full-stack SDR project. This project will serve as the primary talking point in technical interviews, proving you can handle the product life-cycle from research prototype to embedded device.

The Mission: Acquire a bladeRF or LimeSDR board. Modify its open-source FPGA image to include a custom digital hardware block, and route the processed data to a custom C++ application.

1. The Custom IP Core Write a Verilog module for a specialized task (e.g., a burst detector or custom decimator). Write exhaustive Verilator testbenches to prove its timing stability and mathematical accuracy.	2. Integration & Synthesis Instantiate your module within the existing bladeRF/LimeSDR FPGA image. Re-compile the image, ensuring all clock domains and AXI streaming interfaces are correctly mapped.
3. Data Exfiltration Ensure the hardware-accelerated output correctly feeds into the board's USB 3.0 or PCIe data pipeline, making it available to the host processor without packet loss.	4. Lab Validation Use a logic analyzer (or internal SignalTap/ILA cores) alongside an oscilloscope to validate that physical signals match your software simulations exactly.